Object Oriented Programming: Python BootCamp

HIT-emiyon BSc Program
April-May, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 05 : Object Oriented Programming

OOP, or Object-Oriented Programming, is a method of structuring a program by bundling related **properties** and **behaviors** into individual **objects**.

Classes define a type.

Primitive data structures—like numbers, strings, and lists are designed to represent simple things, such as the cost of something, the name of a poem, and your favorite colors, *respectively*. What if you want to represent something much more complicated?

For example, let's say you wanted to track employees in an organization. You need to store some basic information about each employee, such as their name, age, position, and the year they started working.

One way to do this is to represent each employee as a list:

```
Eric = ["Eric", 34, "CEO", 2016]
Alice = ["Alice", "Science Officer", 2017]
```

There are a number of issues with this approach:

- You need to remember that the ith element of the list is the employee's name.
- What if not every employee has the same number of elements in the list? e.g. Age is missing for Alice.

and finally,

- What if you want to define some behaviors for you employees.

This is where classes come into play.

Creating Classes

Classes are used to create user-defined **data structures**.

All class definitions start with the class keyword, which is followed by the name of the class and a colon (:).

Here is an example of a simple Employee class:

```
class Employee:
    pass
```

The body of the Employee class consists of a single statement: the **pass** keyword. **pass** is often used as a place holder where code will eventually go. It allows you to run this code without throwing an error.

Unlike functions and variables, the convention for naming classes in Python is to use **CamelCase** notation, starting with a capital letter.

You can define properties, or attributes for your class. All Employee objects instantiate from your class will then have these properties.

For that, you need to define a special method called **__init__()**

This method is run every time a new Employee object is created and tells Python what the initial state—that is, the initial values of the object's properties—of the object should be.

```
class Employee:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

The first positional argument of **__init__()** is always a variable that eferences the class instance. This variable is almost universally named **self**.

self.name and self.age are called **iinstance attributes** because they belong to the instance of a class.

This might look kind of strange. The **self** variable is referring to an instance of the Employee class, but we haven't actually created an instance yet. Consider it is as a place holder that is used to build the blueprint.

Remember, the class is used to define the Employee data structure. It does not actually create any instances of individual employees with specific names and ages.

While instance attributes are specific to each object, **class attributes** are the same for all instances:

```
class Employee:
    # Class Attribute
    company_name = "Emiyon"
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Instantiate an Object

After you have written a class, you can use it to instantiate as many **objects** of the newly created type as you want.

While the class is the blueprint, an instance of the class is **an object** built from a class that contains real data.

To instantiate an object, type the name of the class, in the original CamelCase, followed by parentheses containing any values that must be passed to the class's `.__init__()` method.

```
emp1 = Employee("David", 23)
emp2 = Employee("Alice", 35)
```

We have created two new Employee instances whose name is David and is 23 years old, and another named Alice who is 35 years old.

```
emp1
```

The number 0x00aeff70 is the address of this Employee object in your computer's memory, and the number you see on your computer will be different.

```
emp1 == emp2
```

```
type(emp1)
```

After the Employee instances are created, you can access their instance attributes by using **dot notation**.

```
emp1.name
```

```
emp2.age
```

Class attributes are accessed the same way:

```
emp1.company_name
```

Both instance and class attributes can be modified dynamically:

```
emp1.age = 36
```

The important takeaway here is that custom objects are mutable by default. Recall that an object is mutable if it can be altered dynamically.

Instance Methods

Classes also have special functions, called methods, that define behaviors and actions that an object created from the class can perform with its data.

```
class Employee:
    company_name = "Emiyon"
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Instance method
    def description(self):
        return f"{self.name} is {self.age} years old"

    # Another instance method
    def update_age(self, new_age):
        self.age = new_age
```

The first argument of an instance method is always **self**.

Let's see how instance methods work in practice,

You can access instance methods by using **dot notation** as well.

```
emp1 = Employee("David", 23)
```

```
emp1.description()
```

```
emp1.update_age(24)
```

```
emp1.description()
```

Let's see what happens when you print() the an Employee object:

```
print(emp1)
```

You can change what gets printed by defining a special instance method called `.__str__()`.

```
class Employee:
    company_name = "Emiyon"
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"
```

```
emp1 = Employee("David", 23)
```

```
print(emp1)
```

```
emp1.name
```

Methods like .__str__() are commonly called **dunder methods** because they begin and end with double underscores.

There are a number of dunder methods available that allow your classes to work well with other Python language features.

Advanced OOP concepts

Inherit From Other Classes

You should now have a pretty good idea of how to create a class that stores some data and provides some methods to interact with that data and define behaviors for an object.

In the next section, you'll see how to take your knowledge one step further and create classes from other classes.

Inheritance is the process by which one class takes on the attributes and methods of another.

Newly formed classes are called **child classes**, and the classes that child classes are derived from are called **parent classes**.

Child classes can override and extend the attributes and methods of parent classes. In other words, child classes inherit all of the parent's attributes and methods but can also specify different attributes and methods that are unique to themselves, or even redefine methods from their parent class.

An example:

Suppose now that you want to model an entire big organization employees with Python classes.

The Employee class you wrote in the previous section can distinguish employee by name and age, but not by department.

You could modify the Employee class by adding a **.dep** attribute:

```
class Employee:
    company_name = "Emiyon"
    def __init__(self, name, age, dep):
        self.name = name
        self.age = age
        self.dep = dep

    def __str__(self):
        return f"{self.name} is {self.age} years old and he is working in {self.dep} department"
```

```
emp1 = Employee("David", 23, "HR")
```

```
print(emp1)
```

We now that each group of employee has slightly different behaviors (tasks to perform). You can simplify the experience of working with the Employee class by creating a child class for each group of employees.

Parent Classes vs Child Classes

```
class Employee:
    company_name = "Emiyon"
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"
```

Remember, to create a child class, you create new class with its own name and then put the name of the parent class in parentheses.

The following creates two new child classes of the Employee class and gives each employee its own sound:

```
class HREmployee(Employee):

    def __init__(self, name, age, wh):
        super().__init__(name,age)
        self.working_hours = wh

    def update_working_hours(self, new_hour):
        self.working_hours  = new_hour

    def __str__(self):
        return f"{self.name} is {self.age} years old and works in HR and has {self.working_hours } working hours!"
```

```
class ManagementEmployee(Employee):

    def __init__(self, name, age):
        super().__init__(name,age)
        self.emp_list = []

    def add_employee(self, emp_name):
        self.emp_list.append(emp_name)

    def give_emp_list(self):
        for i in range(len(self.emp_list)):
            print(f"{i} : {self.emp_list[i]}")
```

With the child classes defined, you can now instantiate some employees of specific section:

```
hr_emp_1 = HREmployee("David", 23, 8)
```

```
hr_emp_1.update_working_hours(10)
```

```
print(hr_emp_1)
```

```
hr_emp_1.working_hours
```

```
management_emp_1 = ManagementEmployee("Alice", 35)
```

```
print(management_emp_1)
```

```
management_emp_1.give_emp_list()
```

```
management_emp_1.add_employee("John")
management_emp_1.add_employee("Maria")
```

```
management_emp_1.give_emp_list()
```

Properties

Another concept from the encapsulation paradigm are **read-only attributes**.

You use it if you think it is a bad idea if someone changes the value of that attribute from outside the class. But you still want the possibility to see the value of the attribute.

This is done by using a getter-function that will return the value of the attribute. If you include the "**@property**" decorator, you can change the behaviour of the function so that it will be more like an attribute (you will not need the brackets for calling the function):

```
class MyClass:

    def __init__(self):
        self._hidden = 42

    @property
    def hidden(self):
        return self._hidden

    def get_hidden(self):
        return self._hidden
```

```
my_object = MyClass()
```

```
print(my_object.hidden)
```

```
# you cannot assign a value to a function, so this line will give an error
my_object.hidden = 0
```

```
# but accessing the value directly still works, although you probably should not do it
my_object._hidden = 0
```

```
# the getter-function has to be called with the brackets
print(my_object.get_hidden())
```

EXERCISES 04 – PART ONE & TWO

```
```